

→ Package in Java :

A package in java is a collection of related classes, interfaces, and sub-packages.

It is mainly used to:

- Organize java programs
- Avoid name conflicts
- Provide access control
- Make code easier to maintain and reuse

→ Why do we need packages?

Suppose we have a large project containing hundreds of classes.

Without package:

Student.java
 Teacher.java
 Employee.java
 Car.java
 Bank.java
 Account.java
 ...

It become difficult to organize.

With package:

College

```

    |
    |→ Student
    |→ Teacher
    |→ Course
    
```

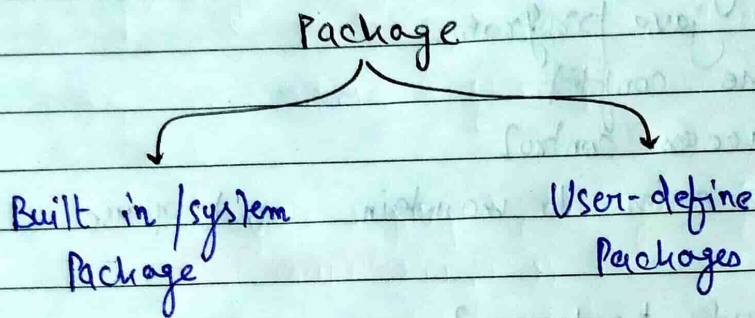
Bank

```

    |
    |→ Account
    |→ Customer
    |→ Transaction
    
```

So package provides a logical organization of classes.

→ Types of package :



(i) Built in / System Package :
 These are package already provided by java :

Eg; java.util
 java.lang
 java.sql
 .etc

• Java.lang :

Eg :

String
 System
 Math
 Object
 Integer
 Double

Java.lang is automatically imported by java.

java.util :

Contains utility and collection classes:

Eg;

Scanner
ArrayList
HashMap
Collections
Arrays

Eg;

```
import java.util.Scanner;
```

(ii) User - define package:

A package created by the programmer is called a user-defined package.

Eg;

mypackage

we can put our own classes inside it

→ Creating a package:

Use the **package** keyword.

```
package mypackage;
```

```
class Student {
```

```
    void display() {  
        cout << "Student";
```

```
    }
```

```
}
```

Note:

The **package** statement should appear at the **beginning** of the **source file**, before imports and class declarations.

→ Importing a package/class:

If a class is in another package, we can use the **import** statement.

Syntax:

```
import packageName . ClassName;
```

e.g;

```
import java.util.Scanner;
```

Now we can use:

```
Scanner sc = new Scanner(System.in);
```

→ Importing multiple classes:

Suppose we need ArrayList and Scanner.

```
import java.util.ArrayList;
import java.util.Scanner;
```

→ Importing all classes from a package:

We can use *****.

```
import java.util.*;
```

This means

import all accessible classes/interfaces directly inside **java.util**.

→ Fully Qualified class Name:

Instead of using import, we can directly specify the directly complete package name.

Eg;

```
java.util.Scanner sc = new new java.util.Scanner(System.in);
```

This is called the **fully qualified name**.

→ User-Defined package example:

→ file 1: Student.java

Package college;

public class Student {

public void display () {

System.out.println("Student class");

}

The class belongs to :

College

→ file 2: Main.java

import college.Student

Class Main {

public static void

main (String args[]) {

Student s = new Student ();

s.display ();

}

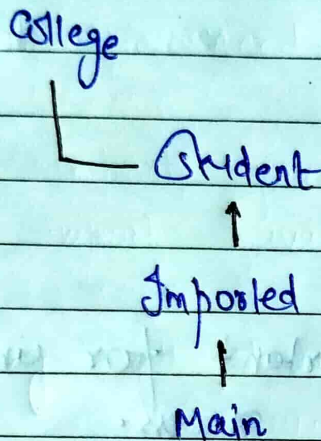
Syntax:

import packageName.className

Output:

Student class

Here:



difference b/w package & import:

<u>Package</u>	<u>import</u>
Defines when package a class belongs to	Allows us to use a class from another package
Used to create/declare package membership	Used to access class without writing full name
Written at beginning of source file	Written after package declaration
Eg; package college;	Eg; import college.Student;

→ Packages and Access Modifiers :

Package are also related to access control.

Java has four access levels :

Private
default
protected
public

Package is particularly important for understanding default and protected access.

→ Default Access :

If no access modifier is specified :

```
class Student {  
    void display () {  
    }  
}
```

The member has default / package - private access.

It can be accessed by classes in the same package.

→ Protected and Packages:

A protected member can be accessed:

- Within the same package
- In subclass in other package, subject to java's inheritance access rules.

~~Main~~

→ Public and packages:

A public class/member can be accessed from other packages, provided the class/member is otherwise accessible.

g:

Package college;

public class Student {

public void student display() {

System.out.println("Hello");

}

}

Another package can import and use it.

— Advantage of Package :

1. Organization :

Related classes are grouped together.

2. Avoid Name Conflicts :

Two packages can contain classes with the same name.

Eg;

College.Student
 School.Student

3. Access Protection :

Package help control access to class and member.

4. Code Reusability :

Classes can be imported and reused in other programs.

5. Easy Maintenance :

Large projects can be divided into logical modules.

[Handwritten signature]